

```

import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Configuração visual dos gráficos
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)

def limpar_tela():
    os.system("cls" if os.name == "nt" else "clear")

def obter_input_valido(prompt, tipo, min_val=None, max_val=None):
    """Garante que o usuário insira um dado válido dentro dos limites
    técnicos."""
    while True:
        try:
            valor = tipo(input(prompt).replace(",", "."))
            if min_val is not None and valor < min_val:
                print(f"❌ Erro: O valor mínimo permitido é
{min_val}.")
                continue
            if max_val is not None and valor > max_val:
                print(f"❌ Erro: O valor máximo permitido é
{max_val}.")
                continue
            return valor
        except ValueError:
            print("❌ Erro: Entrada inválida. Por favor, digite um
número válido.")

def obter_tag_unica(prompt, tags_existentes):
    """Garante que cada bomba tenha um TAG exclusivo e não vazio."""
    while True:
        tag = input(prompt).strip().upper()
        if not tag:
            print("❌ Erro: O TAG do ativo não pode ser vazio.")
            continue
        if tag in tags_existentes:

```

```

        print(f"❌ Erro: O TAG '{tag}' já foi cadastrado. Use um
identificador único.")
        continue
    return tag

def classificar_criticidade(rendimento):
    """Aplica as regras de negócio para a matriz de criticidade."""
    if rendimento > 85:
        return "Baixo Risco (Operação OK)"
    elif 70 <= rendimento <= 85:
        return "Médio Risco (Atenção/Degradação)"
    else:
        return "Alto Risco (Ineficiente/Risco de Falha)"

def gerar_dashboards(df):
    """Gera e exibe os 3 gráficos solicitados na especificação
técnica."""
    print(f"\n📊 Gerando Dashboards Visuais...")

    # Criando uma figura combinada (Grid de 2x2 para o Dashboard)
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))
    fig.suptitle("Dashboard de Desempenho e Custos da Frota de Bombas",
fontsize=18, fontweight='bold', y=0.98)

    # Cores padronizadas para criticidade
    cores_criticidade = {
        "Baixo Risco (Operação OK)": "#2ecc71",      # Verde
        "Médio Risco (Atenção/Degradação)": "#f1c40f", # Amarelo
        "Alto Risco (Ineficiente/Risco de Falha)": "#e74c3c" # Vermelho
    }

    # --- GRÁFICO 1: Vazão Nominal vs. Vazão Real (Barras Justapostas)
    ---
    x = np.arange(len(df["TAG"]))
    largura = 0.35
    axes[0, 0].bar(x - largura/2, df["Vazão Nominal (m³/h)"], largura,
label="Nominal", color="#3498db")
    axes[0, 0].bar(x + largura/2, df["Vazão Real (m³/h)"], largura,
label="Real", color="#e67e22")
    axes[0, 0].set_title("Comparativo de Capacidade: Vazão Nominal vs.
Real", fontsize=12, fontweight='bold')

```

```

axes[0, 0].set_ylabel("Vazão (m³/h)")
axes[0, 0].set_xticks(x)
axes[0, 0].set_xticklabels(df["TAG"], rotation=45)
axes[0, 0].legend()

# --- GRÁFICO 2: Ranking de Custo Operacional Diário ---
df_ordenado_custo = df.sort_values(by="Custo Diário (R$)",
ascending=False)
sns.barplot(x="Custo Diário (R$)", y="TAG", data=df_ordenado_custo,
ax=axes[0, 1], palette="Purples_r")
axes[0, 1].set_title("Ranking de Custo Operacional Diário
(R$/dia)", fontsize=12, fontweight='bold')
axes[0, 1].set_xlabel("Custo Diário (R$)")
axes[0, 1].set_ylabel("TAG da Bomba")

# --- GRÁFICO 3: Distribuição por Criticidade ---
contagem_criticidade = df["Criticidade"].value_counts()
# Mapeia as cores conforme as categorias que realmente existem no
subset de dados atual
cores_pizza = [cores_criticidade[cat] for cat in
contagem_criticidade.index]

axes[1, 0].pie(contagem_criticidade,
labels=contagem_criticidade.index, autopct='%1.1f%%',
startangle=140, colors=cores_pizza,
wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[1, 0].set_title("Distribuição da Frota por Nível de
Criticidade", fontsize=12, fontweight='bold')

# Deletando o quarto subplot vazio para deixar o layout limpo
fig.delaxes(axes[1, 1])

plt.tight_layout()
plt.show()

def exportar_dados(df):
    """Permite ao usuário exportar os dados estruturados."""
    escolha = input("\n📄 Deseja exportar os resultados? (S - Sim / N -
Não): ").strip().upper()
    if escolha == 'S':
        print("1 - Exportar para Excel (.xlsx)")
        print("2 - Exportar para CSV (.csv)")

```

```

formato = input("Escolha o formato (1 ou 2): ").strip()

nome_arquivo = input("Digite o nome do arquivo (pressione Enter
para 'relatorio_frota_bombas'): ").strip()
if not nome_arquivo:
    nome_arquivo = "relatorio_frota_bombas"

if formato == '1':
    df.to_excel(f"{nome_arquivo}.xlsx", index=False)
    print(f"✅ Arquivo salvo com sucesso como
'{nome_arquivo}.xlsx' no diretório atual!")
else:
    df.to_csv(f"{nome_arquivo}.csv", index=False, sep=";",
encoding="utf-8-sig")
    print(f"✅ Arquivo salvo com sucesso como
'{nome_arquivo}.csv' no diretório atual!")

def main():
    limpar_tela()
    print("="*65)
    print("          SISTEMA DE ANÁLISE DE DESEMPENHO DE BOMBAS CENTRÍFUGAS
")
    print("="*65)

    num_bombas = obter_input_valido("Informe a quantidade de bombas (N)
a analisar: ", int, min_val=1)

    dados_bombas = []
    tags_cadastradas = set()

    # Loop de coleta de dados
    for i in range(num_bombas):
        print(f"\n--- Inserindo Dados do Ativo {i+1} de {num_bombas}
---")
        tag = obter_tag_unica(f"TAG ou Setor da Bomba: ",
tags_cadastradas)
        tags_cadastradas.add(tag)

        q_nom = obter_input_valido(f"Vazão Nominal de Projeto (m³/h):
", float, min_val=0.01)
        p_nom = obter_input_valido(f"Potência Nominal do Motor (kW): ",
float, min_val=0.01)

```

```

rendimento = obter_input_valido(f"Rendimento Atual da Bomba η
(%): ", float, min_val=1.0, max_val=100.0)
horas = obter_input_valido(f"Horas de Operação Diária (h): ",
float, min_val=0.1, max_val=24.0)
custo_energia = obter_input_valido(f"Custo da Energia Elétrica
(R$/kWh): ", float, min_val=0.01)
custo_mnt = obter_input_valido(f"Custo de Manutenção Preventiva
(R$/h): ", float, min_val=0.0)

# --- PROCESSAMENTO / REGRAS DE ENGENHARIA ---
# Nota de Engenharia: Correção da potência real consumida
baseada na eficiência
consumo_diario = (p_nom / (rendimento / 100.0)) * horas
custo_diario = (consumo_diario * custo_energia) + (horas *
custo_mnt)
vazao_real = q_nom * (rendimento / 100.0)
criticidade = classificar_criticidade(rendimento)

# Guardando em formato de dicionário para posterior DataFrame
dados_bombas.append({
    "TAG": tag,
    "Vazão Nominal (m³/h)": q_nom,
    "Vazão Real (m³/h)": vazao_real,
    "Rendimento (%)": rendimento,
    "Consumo Diário (kWh)": consumo_diario,
    "Custo Diário (R$)": custo_diario,
    "Criticidade": criticidade
})

# Construção do Dataframe
df_original = pd.DataFrame(dados_bombas)

# Opção de Filtro Dinâmico antes de exibir resultados
df_exibicao = df_original.copy()
filtrar = input("\n🔍 Deseja filtrar a visualização da tabela
apenas para ativos de 'Alto Risco'? (S/N): ").strip().upper()
if filtrar == 'S':
    df_exibicao = df_original[df_original["Criticidade"] == "Alto
Risco (Ineficiente/Risco de Falha)"]
    if df_exibicao.empty:
        print("💡 Nenhuma bomba encontrada na zona de Alto Risco.
Mostrando frota total.")
    df_exibicao = df_original.copy()

```

```

# --- APRESENTAÇÃO DE RESULTADOS (SAÍDAS) ---
limpar_tela()
print("\n" + "="*85)
print("🔍 INSIGHTS AUTOMÁTICOS DE ENGENHARIA")
print("="*85)

# 1. Identificação do Pior Ativo (Mais crítico)
idx_pior_rendimento = df_original["Rendimento (%)"].idxmin()
pior_bomba = df_original.loc[idx_pior_rendimento]
print(f"⚠️ O Alvo Crítico: Atenção: A bomba [{pior_bomba['TAG']}]
apresentou o pior rendimento da frota ({pior_bomba['Rendimento
(%)']:.1f}%) e deve ser priorizada para manutenção.")

# 2. Custo da Ineficiência
df_criticas = df_original[df_original["Criticidade"] == "Alto Risco
(Ineficiente/Risco de Falha)"]
custo_total_critico = df_criticas["Custo Diário (R$)"].sum()
print(f"💰 Custo da Ineficiência: O impacto operacional diário dos
ativos na zona VERMELHA é de R$ {custo_total_critico:,.2f}/dia
(Potencial de Economia).")
print("="*85)

# Exibição do Painel de Dados estruturado
print("\n📋 PAINEL DE DADOS RESUMO (Tabela de Performance):")
# Formatação para melhor leitura visual na tabela do terminal
df_formatado = df_exibicao.copy()
df_formatado["Vazão Real (m³/h)"] = df_formatado["Vazão Real
(m³/h)"].map("{:.1f}".format)
df_formatado["Consumo Diário (kWh)"] = df_formatado["Consumo Diário
(kWh)"].map("{:.1f}".format)
df_formatado["Custo Diário (R$)"] = df_formatado["Custo Diário
(R$)"].map("R$ {:.2f}".format)
print(df_formatado[["TAG", "Vazão Nominal (m³/h)", "Vazão Real
(m³/h)", "Consumo Diário (kWh)", "Custo Diário (R$)",
"Criticidade"]].to_string(index=False))

# Geração dos Dashboards Visuais
gerar_dashboards(df_original)

# Exportação final dos dados (Sempre usando o dataframe completo
original)
exportar_dados(df_original)

```

```
if __name__ == "__main__":  
    main()
```